1. Introduction

Assignment 7a extends the Assignment 6 Rock-Paper-Scissors implementation into a Qt6 application that supports local text mode, local GUI play, and network multiplayer.

Extra Credit implemented: the GUI includes network host/join play over TCP, with two network modes: VS_COMPUTER and VS_PLAYER. This makes the extra-credit feature easy to identify and demonstrates that the application can run as either a local game or a networked multiplayer game.

This report focuses on the required deliverables from Assignment 7: inversion of control in GUI mode, screenshots of the layout and running application, reuse/refactoring from Assignment 6, and run instructions for text mode and GUI mode.

---

2. Requirements and Design Goals

The assignment requirements fall into two groups: functional behavior and software design quality.

The functional GUI requirements are:

1. Allow users to choose the computer strategy (random or smart).
2. Allow users to set the number of game rounds.
3. Allow users to enter the human move each round.
4. Display the current round number.
5. Display the human move for the round.
6. Display the computer prediction of the human move.
7. Display the computer move.
8. Display who won the round or whether it was a tie.
9. Display cumulative wins and ties after each round.

The design goals were:

1. Keep one executable that can run in text mode or GUI mode.
2. Reuse the Assignment 6 game logic instead of rewriting it inside the GUI.
3. Use event-driven GUI behavior instead of procedural polling.

For Assignment 7a, we added a further goal: support host/join network play without changing the base rules of Rock-Paper-Scissors. The network code should act as an adapter around the existing game engine, not a replacement for it.

---

3. System Architecture

3.1 Layered decomposition

The implementation is organized into layers with clear responsibilities.

1. Domain layer: Choice values, round outcomes, and round result data.
2. Strategy layer: Strategy interface, RandomStrategy, and SmartStrategy.
3. Player layer: HumanPlayer, GUIHumanPlayer, and ComputerPlayer.
4. Engine layer: GameEngine for shared game flow, GUIGameEngine for local GUI play, and NetworkGameEngine for network play.
5. Network layer: NetworkManager for TCP communication and protocol parsing.
6. Presentation layer: MainWindow and the Qt UI file.
7. Persistence layer: FrequencyStore for learning data.

3.2 Core abstractions

GameEngine centralizes the reusable round lifecycle and score bookkeeping. GUIGameEngine inherits from this design and converts game events into Qt signals so the GUI can update itself. NetworkGameEngine handles server-side round coordination for network sessions and emits signals for live display. NetworkManager is responsible for sockets, TCP connections, message framing, and protocol events.

3.3 Architectural rationale

The architecture follows three principles.

1. Single responsibility: each class owns one main concern.
2. Open/closed behavior: strategies and modes are added by extension.
3. Interface-driven collaboration: high-level code depends on abstractions, not widget details or raw socket calls.

---

4. Inversion of Control in GUI Mode

Inversion of control is the central difference between console execution and GUI execution.

4.1 Control flow in text mode

Text mode still uses a synchronous loop:

1. Initialize the game.
2. Read the player's input.
3. Compute the round result.
4. Print the output.
5. Save learning data when the game ends.

In this mode, application code owns the flow of control.

4.2 Control flow in local GUI mode

In GUI mode, Qt owns the call sequence through its event loop:

1. MainWindow is created.
2. The user clicks buttons or changes controls.
3. Slots in MainWindow react to those signals.
4. GUIGameEngine processes the round and emits result signals.
5. MainWindow updates labels, images, history cards, and score displays.

The application does not poll for input. Instead, Qt calls into the application when events occur.

4.3 Control flow in network GUI mode

Network play follows the same event-driven pattern, but the events may come from the network instead of only from the local UI. In this version, MainWindow reacts to both button clicks and NetworkManager signals.

1. The host or client starts a connection.
2. NetworkManager sends or receives protocol messages.
3. NetworkGameEngine updates round state when both choices are available.

4.4 Engineering consequences

Because the GUI is event-driven, the application has to manage state carefully. Controls are enabled or disabled depending on whether the user is waiting, playing locally, or participating in a network game. This prevents invalid actions such as changing the strategy after a round has already started.

---

5. Reuse from Assignment 6 and Refactoring Strategy

5.1 Reused components

The following parts were preserved from Assignment 6:

1. Choice representation and round outcome logic.
2. Strategy interface and both strategy implementations.
3. ComputerPlayer delegation.
4. FrequencyStore persistence.
5. The core round/score logic in GameEngine.

This reuse reduced risk and ensured that the same game rules apply in all modes.

5.2 Required refactoring for dual-mode support

To support local GUI mode, the project split presentation behavior from game logic. GUIGameEngine was added so round results could be emitted as Qt signals instead of printed directly. GUIHumanPlayer was added so the GUI could provide player input

without using console I/O. main.cpp was updated to choose between GUI mode and text mode based on command-line arguments.

5.3 Refactoring for network multiplayer

Assignment 7a adds a second layer of refactoring for network play.

1. NetworkManager wraps QTcpServer and QTcpSocket so the GUI does not need raw socket code.
2. NetworkGameEngine acts as the server-side controller for network rounds.
3. MainWindow gained a network panel so the user can host or join a game without leaving the GUI.
4. The same core strategy and rule logic still applies, but now the source of the opponent move may be a remote client instead of only the local AI.

This approach keeps the network code as an adapter. It does not replace the game rules; it only transports input and output between machines.

5.4 Change log from Assignment 6 to Assignment 7a

The following differences are based on implemented code, not planned features.

1. Build/runtime platform: Assignment 6 uses a Makefile and console execution only; Assignment 7a uses CMake with Qt6 Widgets and Qt6 Network.
2. Entry behavior: Assignment 7a adds GUI startup (`--gui`) and defaults to GUI when no text-mode flag is provided.
3. Presentation layer: Assignment 7a adds MainWindow/MainWindow.ui and GUI adapters (GUIGameEngine, GUIHumanPlayer) for button-driven play.
4. Network layer: Assignment 7a adds NetworkManager and NetworkGameEngine with implemented messages JOIN, WELCOME, GAME_START, ROUND_START, CHOICE, RESULT, and GAME_OVER.
5. Multiplayer modes: Assignment 7a supports VS_COMPUTER and VS_PLAYER; VS_PLAYER uses two listeners (port and port+1) because each NetworkManager accepts one client.
6. Reuse preserved: core rules, strategies, scoring, and FrequencyStore persistence remain compatible across console, local GUI, and network workflows.

---

6. Network Multiplayer Design

This section is the main addition in Assignment 7a.

6.1 Roles and modes

The network version supports two game modes:

1. VS_COMPUTER: one client plays against the server's AI.
2. VS_PLAYER: two human clients play against each other and the server acts as referee.

This design gives the host a choice between a human-vs-AI game and a human-vs-human game, without changing the overall UI layout.

6.2 Network runtime implementation

Network gameplay is split into two cooperating components.

1. NetworkManager handles TCP I/O, host/client connections, newline-delimited message framing, and protocol parsing.
2. NetworkGameEngine is the host-side round coordinator that waits for choices, resolves winners, updates scores, and emits UI updates.

The implemented protocol messages are:

1. JOIN:
2. WELCOME:
3. GAME_START::<RANDOM|SMART>
4. ROUND_START:
5. CHOICE:<R|P|S>
6. RESULT::::::
7. GAME_OVER:::

Each round follows a fixed sequence: ROUND_START -> CHOICE -> RESULT, then GAME_OVER after the final round. This keeps game state synchronized while keeping game-rule decisions inside the engine.

6.3 Host workflow

When the user hosts a game, MainWindow does the following:

1. Reads the selected port and round count.
2. Starts one NetworkManager listener on the selected port.
3. In VS_PLAYER mode, starts a second NetworkManager listener on port+1 so a second client can join.
4. Chooses VS_COMPUTER or VS_PLAYER.
5. Creates NetworkGameEngine.
6. Connects engine and network signals to UI slots.
7. Locks the network controls while the game is active.

For VS_COMPUTER, the host waits for one remote player on the selected port. For VS_PLAYER, the two remote players connect separately through the two listener ports shown in the UI.

6.4 Join workflow

When the user joins a game, MainWindow creates a client-side NetworkManager, connects to the host IP and port entered in the UI, and waits for the server to send GAME_START and ROUND_START messages. After that, the local Rock/Paper/Scissors buttons are repurposed so the client sends CHOICE messages over the network instead of directly calling the local engine.

Failure handling is implemented at the UI level: host-start failure shows an error dialog, client connection failure shows a warning and restores controls, and disconnect events clear active network objects so users can start a new session.

The current implementation also includes an 8-second client connection timeout and a safe teardown order at game completion to avoid silent hangs or re-entrant cleanup crashes.

---

7. GUI Design Rationale

7.1 Layout overview

The MainWindow layout has three major areas:

1. Game setup and strategy selection.
2. Network multiplayer controls and status.
3. Round display, score display, and round history.

This makes the interface useful for both local play and network play.

7.2 Interaction constraints

The interface disables and enables controls based on state.

1. Local game buttons are disabled until a local game begins.
2. Network controls are disabled while hosting or joining.
3. Disconnect becomes available only after a connection is active.
4. When a session ends, the controls are restored.

This prevents users from changing modes in the middle of a round and keeps the GUI consistent with the engine state.

7.3 Visual communication

The GUI uses hand images, status messages, round labels, winner labels, and score labels to make each round easy to understand. The round history panel gives the user a compact timeline of what happened during the game.

In network mode, the status area is especially important because it tells the user whether they are hosting, waiting, connecting, or playing.

---

8. Execution Instructions

This section describes how to run the application in the modes required by the assignment.

8.1 Prerequisites

1. CMake 3.16 or higher.
2. A C++17 compiler.
3. Qt6 Widgets and Qt6 Network development libraries.

8.2 Build procedure

```
cmake -S . -B build
cmake --build build
```

8.3 Text mode

Random strategy:

```
./build/rps --random --rounds=20
```

Smart strategy:

```
./build/rps --smart --rounds=20 --n=5
```

8.4 Local GUI mode

GUI mode is the default when no text-mode flags are provided:

```
./build/rps
```

You can also launch it explicitly:

```
./build/rps --gui
```

8.5 Network mode workflow

There are two ways to start network play.

VS_COMPUTER mode uses 2 running instances:

1. Start instance 1 and choose Network Multiplayer > Host Game.
2. Select `VS_COMPUTER`, choose a round count, and keep the port value shown in the host panel.
3. Start instance 2 and choose Network Multiplayer > Join Game.
4. Enter the host IP and the same port value used by the host.
5. Wait for the host to report that one client joined.
6. When the host starts the game, use the Rock/Paper/Scissors buttons in the client window to play.

VS_PLAYER mode uses 3 running instances:

1. Start instance 1 and choose Network Multiplayer > Host Game.
2. Select `VS_PLAYER`, choose a round count, and note the host port shown in the panel.
3. Start instance 2 and join the host IP on the host port.
4. Start instance 3 and join the same host IP on `host port + 1`.

5. Wait until the host shows that both players are connected.
6. When each round starts, both client windows choose Rock, Paper, or Scissors.
7. The host window shows the round result, scores, and game-over state.
8. After the game ends, disconnect or close the windows before starting a new session.

In both modes, the host determines whether the session is `VS_COMPUTER` or `VS_PLAYER`. In `VS_COMPUTER` mode, the strategy combo is used by the host-side engine; in `VS_PLAYER` mode, the strategy selection is not used because both sides are remote players.

8.6 Suggested network demonstration

Run one `VS_COMPUTER` session and one `VS_PLAYER` session, then verify that `ROUND_START`, `RESULT`, and `GAME_OVER` update the GUI correctly and that disconnecting restores the controls for a new session.

---

9. Screenshots

Figure 1. Qt Designer Layout

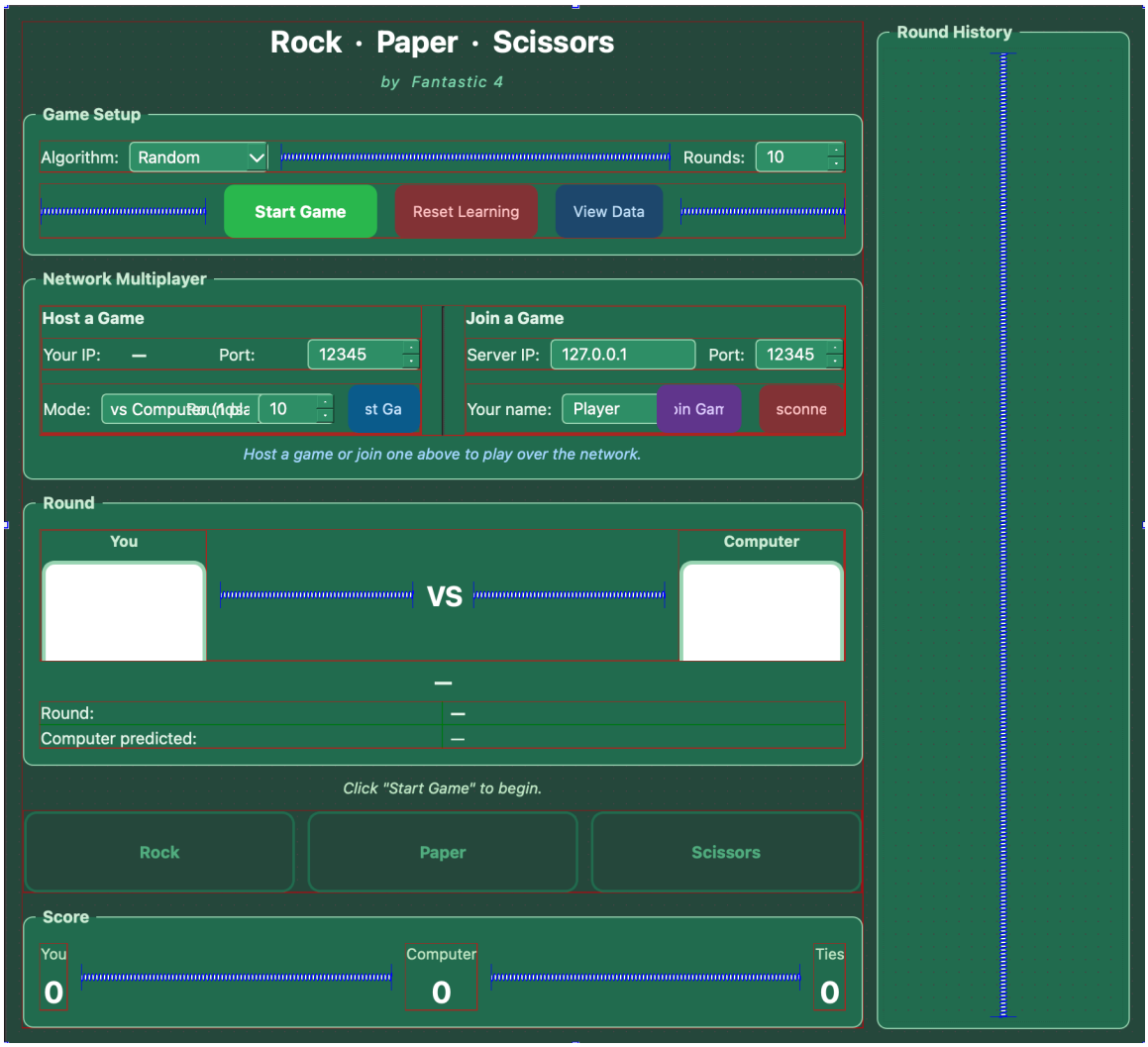
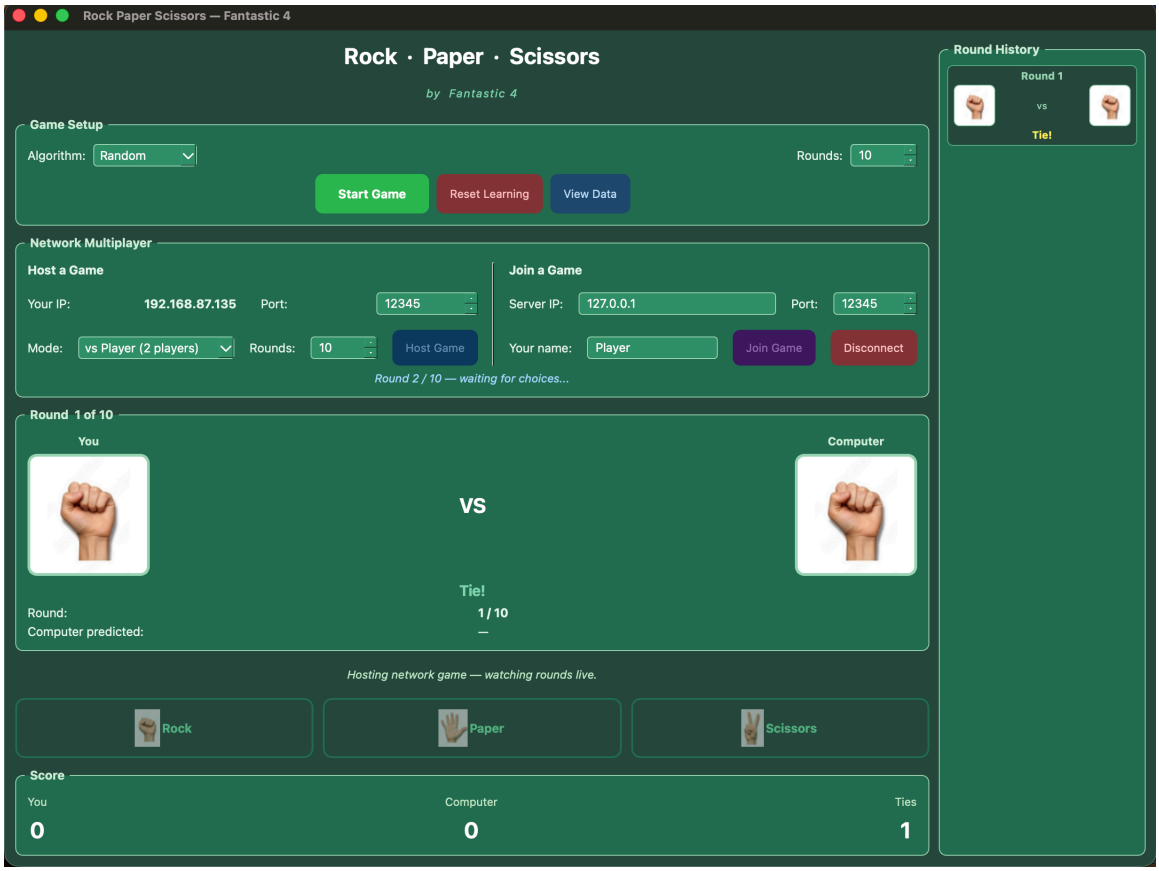
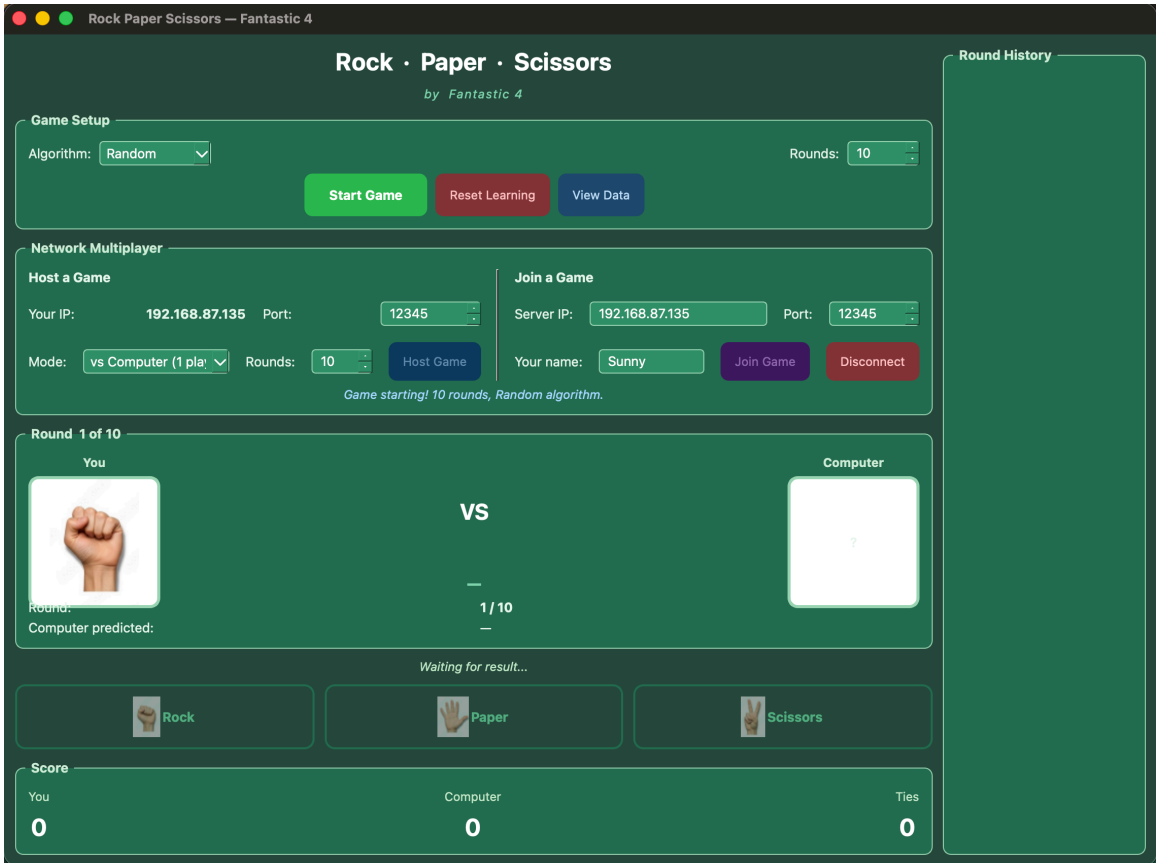


Figure 2. Application During Gameplay

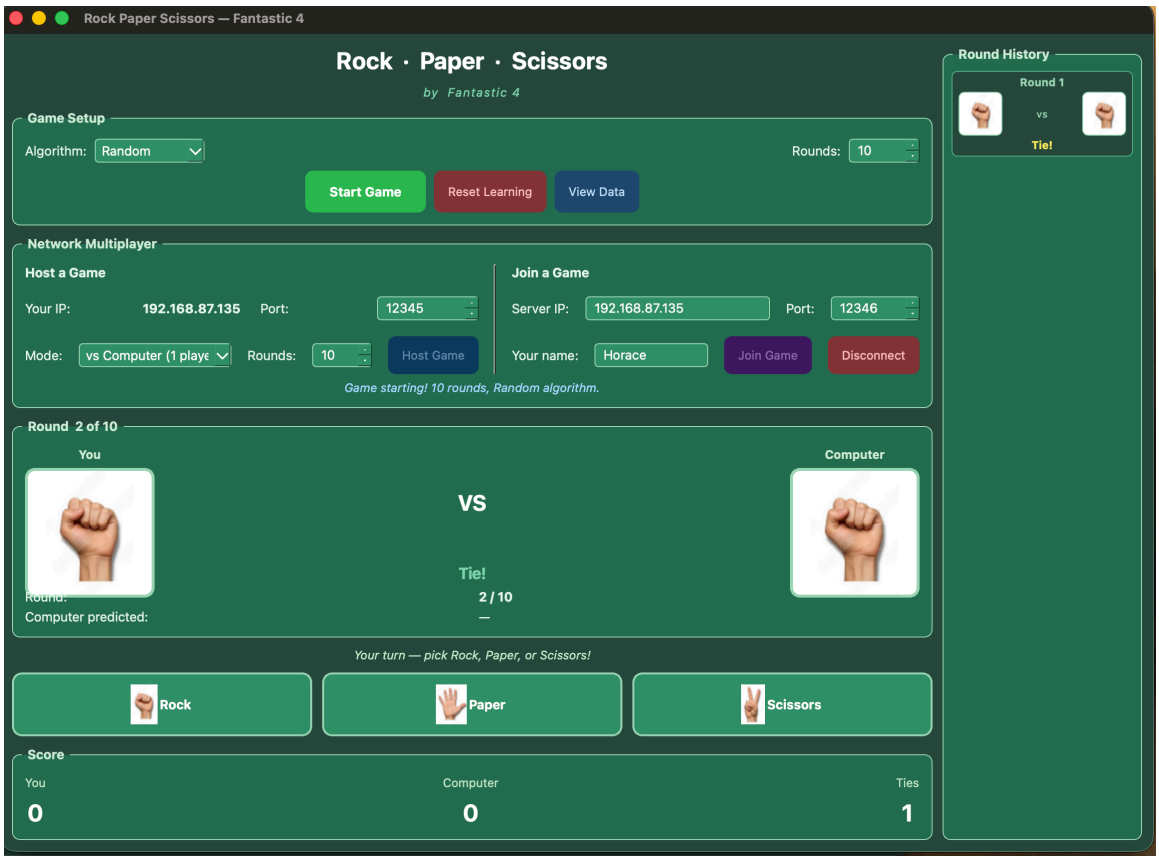
Host view:



Player 1 view:



Player 2 view:



11. Limitations and Future Work

The current implementation meets the assignment goals, but it still has practical limits.

1. The protocol is intentionally simple and optimized for clarity rather than scale.
2. Switching between VS_COMPUTER and VS_PLAYER requires starting a new session.
3. Smart strategy quality depends on observed player patterns.

Future work includes richer lobby/session management and automated protocol-level tests.

---

12. Conclusion

Assignment 7a extends the earlier Rock-Paper-Scissors work into a more capable Qt6 system with both local and networked play. The final design keeps the game rules, strategy logic, and score tracking reusable while moving GUI and socket responsibilities into separate adapter classes. This separation allowed us to add host/join multiplayer without rewriting the core game engine. The result is a single application that can run in text mode, local GUI mode, or network multiplayer mode while keeping the codebase organized and maintainable.